Green DevOps
Research Framework

Table of Contents

Release Stage ***3***

What this component does 3

Key Capabilities 3

Thermal and Cooling Awareness (Built-in Constraint) 4

Outputs of the Release Stage 4

Why this matters 4

Deploy Stage ***5***

Sub Component 1: Deployment Energy Profiler **5**

Sub Component 2: AI-Based Green Deployment Decision Engine **6**

Operation Stage ***7***

Research Overview / Objective **7**

Novelty / Research Contribution **7**

Research Methodology **7**

Monitoring Stage ***10***

Purpose 10

What this component does 10

Core Responsibilities 10

How it works (High-Level Flow) 11

Outputs of the Monitoring Stage 11

Why this matters 12

Diagram ***12***

Summary ***13***

Release Stage

The Release stage acts as a **decision and control layer** that determines *what* gets released, *how much verification is needed*, and *when release activities should run* to minimize energy use, carbon emissions, and cooling overhead—without compromising software quality.

What this component does

Before a release is finalized, the system evaluates sustainability impact and makes informed decisions instead of blindly executing compute-heavy pipelines.

It focuses on three core actions:

- **Reduce unnecessary release work**
- **Optimize verification effort**
- **Schedule release activities more sustainably**

Key Capabilities

1. Release Work Optimization

The component avoids wasteful computation by:

- Reusing existing build artifacts where safe
- Performing incremental builds instead of full rebuilds
- Using cached dependencies to reduce repeated downloads

This directly lowers CPU usage and prevents avoidable energy consumption during release preparation.

---

2. Intelligent Release Verification

Instead of running all tests by default, verification is executed **selectively**.

The system:

- Categorizes tests by cost, risk, and relevance
- Runs lightweight, high-value checks first
- Escalates to heavier tests only when justified

This maintains release confidence while cutting sustained compute load.

---

3. Carbon-Aware Release Scheduling

Non-urgent release tasks are aligned with **lower-carbon energy windows**.

Scheduling decisions consider:

- Release urgency
- Expected workload intensity
- Carbon intensity trends

High-impact workloads are delayed when possible to reduce emissions.

Thermal and Cooling Awareness (Built-in Constraint)

Release pipelines are treated as **heat-generating events**, not just software processes.

To avoid inefficient cooling behavior:

- CPU-intensive tasks are staged instead of stacked
- Parallelism is limited during heavy workloads
- Short cool-down gaps are introduced between verification stages

This reduces thermal spikes and lowers cooling energy demand.

Outputs of the Release Stage

- Optimized release execution plan
- Reduced compute and cooling energy usage
- Lower carbon emissions during release preparation
- Release decisions informed by energy, carbon, and thermal data

---

Why this matters

The Release stage is a hidden energy hotspot. By turning it into a **sustainability-aware decision layer**, this component achieves measurable environmental benefits while preserving reliability, quality, and delivery confidence.

Deploy Stage

Sub Component 1: Deployment Energy Profiler

Purpose

The Deployment Energy Profiler is responsible for measuring and estimating the energy usage and carbon emissions that occur during the deployment process.

Without measurement, it is impossible to improve environmental efficiency.
This component creates the baseline data required for green optimization.

What this component does

When a deployment starts, the profiler:

- Observes how much CPU is used
- Observes how much memory is used
- Measures deployment duration
- Estimates energy consumption
- Estimates carbon emissions

All this information is recorded **only for the deployment stage**.

How it works

1. Deployment begins
2. Monitoring scripts collect system usage data
3. Energy usage is calculated using standard estimation formulas
4. Carbon emission is estimated based on regional energy intensity
5. Results are stored as deployment energy logs

Output of this component

- Energy used per deployment
- Carbon emission per deployment
- Historical deployment energy data

This output becomes the **input data** for the AI-based decision component.

Sub Component 2: AI-Based Green Deployment Decision Engine

Purpose

This component intelligently decides when and how to deploy software in a way that minimizes environmental impact, while still maintaining system reliability. This component introduces intelligence into the deploy stage.

What this component does

Before deployment is executed, the system thinks first.
It answers two key questions:

1. Is this a good time to deploy from an energy and carbon point of view?
2. Which deployment strategy will consume the least energy for this situation?

The decision is made using:

- Historical deployment energy data (from Component 1)
- Time-based patterns
- Deployment urgency level
- Past deployment outcomes

AI Decision Logic

The AI model evaluates:

- Time of day (peak vs off-peak energy)
- Estimated energy cost of deployment
- Carbon intensity trends
- Deployment type (small update, major release)

Based on this evaluation, it decides:

- Deploy immediately **or**
- Delay deployment to a greener time window
  AND
- Select the most suitable deployment strategy (e.g., rolling, canary, blue-green)

How this component works (Steps)

1. Deployment request is triggered
2. AI engine receives energy data from the profiler
3. AI evaluates timing and strategy options
4. Green deployment plan is generated
5. Deployment is executed using the selected plan

Output of this component

- Selected deployment time
- Selected deployment strategy

- Reduced energy and carbon impact during deployment

Operation Stage

A Carbon-Aware Auto-Scaling Model

Research Overview / Objective

The DevOps Operation Phase is responsible for running, monitoring, scaling, and maintaining live applications. Traditional auto-scaling approaches prioritize performance metrics such as CPU usage, memory, and latency while ignoring environmental sustainability.

The objective of this research is to design a Carbon-Aware Auto-Scaling Model that automatically adjusts resources for live cloud applications based on carbon intensity, predicted workload, and performance requirements, reducing energy consumption and environmental impact while maintaining system reliability.

Novelty / Research Contribution

- Integrates carbon intensity as a first-class metric
- Automatically scales live applications to reduce energy usage
- Uses AI-based workload prediction for proactive scaling
- Balances performance (SLA) and sustainability
- Provides real-time automated actions beyond monitoring or reports

Research Methodology

A. Inputs

- Performance Metrics: CPU, memory, network, latency, request throughput
- Carbon Intensity Data: Cloud region energy mix (renewable vs fossil), kg CO_2 per kWh
- Workload / Traffic Prediction: Historical patterns via AI/ML (ARIMA, LSTM)
- Service Characteristics: Microservice priority, SLA requirements
- Cloud Resource Info: Server types, cost, energy efficiency

B. Process / Workflow

1. Monitoring Layer

- Collects application metrics and carbon intensity data

2. Prediction Layer

- Uses historical traffic with AI models to predict future load

3. Decision Layer

- Computes optimal resource allocation considering predicted workload, SLA, and carbon intensity

4. Auto-Scaling Execution

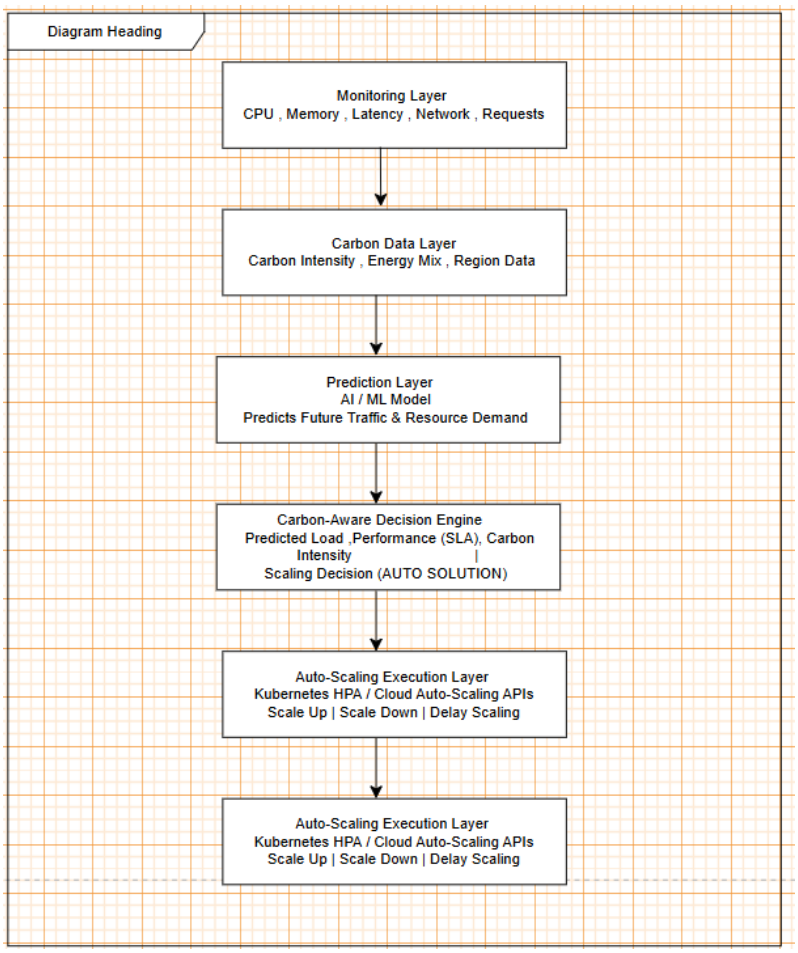
- Dynamically adjusts CPU/RAM limits or container replicas inside the same high-capacity server
- Avoids over-provisioning; incremental resource allocation

5. Feedback Layer

- Monitors energy savings, carbon reduction, and performance
- Refines future predictions and scaling decisions

C. Expected Outcomes

- Reduced carbon footprint
- Optimized energy usage
- Maintained system performance (SLA compliance)
- Dynamic resource optimization in real-time
- Proactive automated actions



Data sets:

https://www.kaggle.com/datasets/zoya77/cloud-workload-dataset-for-scheduling-analysis/data

Monitoring Stage

Purpose

The Monitoring stage provides the **data foundation for Green DevOps** by measuring, estimating, and structuring energy consumption and carbon emissions across CI/CD pipelines. Without this component, sustainability cannot be analyzed, optimized, or justified.

What this component does

During CI/CD execution, the monitoring system **captures raw execution and resource data** and converts it into **usable sustainability intelligence**.

It enables DevOps teams and downstream components to answer:

- How much energy does each pipeline, stage, and job consume?
- Where are carbon emissions coming from?
- Which parts of the pipeline are inefficient or regressing?

Core Responsibilities

1. Sustainability Measurement

- Estimates energy consumption per job, stage, and pipeline run
- Converts energy usage into carbon emissions using regional carbon intensity data

---

2. Resource & Execution Monitoring

- Tracks CPU, memory, disk, and network usage
- Measures execution time, idle time, and parallelism behavior

---

3. Sustainability Attribution

- Maps energy and carbon impact to:
    - Build, test, deploy stages
    - Individual jobs and commits

This makes sustainability **traceable**, not abstract.

---

4. Analysis & Pattern Detection

- Identifies energy-intensive pipeline stages
- Detects anomalies, regressions, and inefficient execution patterns
- Establishes sustainability baselines for normal behavior

---

5. Dataset Creation & Feedback

- Produces structured, labeled datasets ready for:
  - Analysis
  - Optimization
  - Machine learning
- Feeds sustainability data to Release, Deploy, and Operate stages

---

How it works (High-Level Flow)

1. CI/CD jobs execute
2. Resource and execution metrics are collected
3. Energy consumption is estimated using proxy models
4. Carbon emissions are calculated
5. Impact is attributed to stages and jobs
6. Structured datasets and sustainability indicators are generated

---

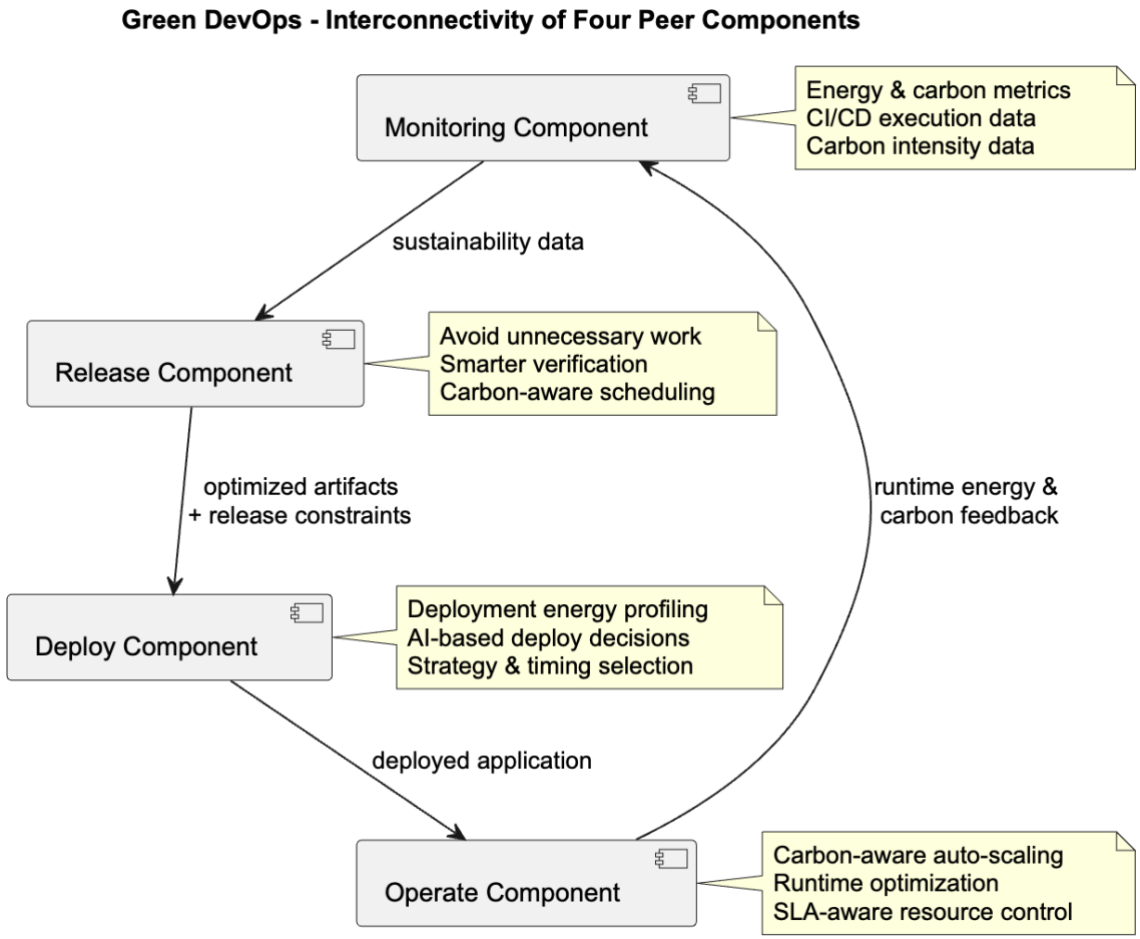
Outputs of the Monitoring Stage

- Energy usage per pipeline, stage, and job
- Carbon emissions per pipeline run
- Sustainability baselines and anomaly signals
- Reusable datasets for decision-making and optimization

Why this matters

Without monitoring, Green DevOps is guesswork.
This component turns sustainability from an **invisible side effect** into a **measurable engineering metric**, enabling evidence-based decisions across the DevOps lifecycle.

Diagram



Summary

This document presented four complementary research components that collectively form an integrated Green DevOps framework: Sustainability-Aware Release Engineering, Green Deployment Optimization, Carbon-Aware Operation and Auto-Scaling, and Sustainability-Aware CI/CD Monitoring. Each component targets a distinct phase of the DevOps lifecycle while addressing a specific source of energy consumption and carbon emissions.

The Release component focuses on preventing unnecessary computation through intelligent artifact reuse, selective verification, carbon-aware scheduling, and thermal-aware pipeline shaping. The Deploy component introduces deployment-stage measurement and AI-driven decision-making to minimize energy and carbon impact during software rollout. The Operation component extends sustainability considerations into runtime environments by dynamically scaling resources based on carbon intensity, workload predictions, and SLA constraints. The Monitoring component provides systematic measurement, attribution, and dataset generation, enabling empirical evaluation, cross-stage feedback, and data-driven optimization across the lifecycle.

Together, these components operate as interconnected peers, exchanging sustainability-relevant data and feedback across stages rather than functioning as isolated optimizations. This integrated approach ensures that sustainability is addressed proactively before execution, optimized during deployment, continuously managed at runtime, and empirically validated through monitoring. By embedding sustainability-aware decision-making throughout the DevOps lifecycle, the proposed framework moves beyond reporting and tooling, positioning sustainability as a measurable, actionable, and continuously improvable software engineering concern.